19 · Bridge to LLMOps

> **Goal:** After this chapter you can map every MLOps control you know onto an LLM application, say precisely what carries over and what changes, and know when to hand off to the sibling LLMOps interview-preparation project for depth.

Table of Contents

- 19.1 What Carries Over from MLOps and What Changes
- 19.2 Evaluating and Monitoring Generative Systems (Short Treatment)
- 19.3 Serving and Cost for Large Models on a Classical ML Platform

19.1 What Carries Over from MLOps and What Changes

Conceptual Deep Dive

The most useful interview framing: **treat the LLM application as a pipeline with an unreliable component**. Everything around the model (versioning, gates, rollouts, SLOs, cost attribution, access control) stays the same discipline; the component inside is non-deterministic, often hosted by someone else, and judged by fuzzy criteria.

MLOps concept	LLM-application equivalent
dataset	prompts, golden sets, document corpora
features	retrieved context, conversation state
model	base model + prompt + tools + sampling settings
model registry	prompt and model registry, index registry
CI tests	eval suites with judges and rubrics
drift monitoring	trace, quality and cost monitoring, query-distribution drift
canary / shadow	same, compared on judged quality and cost

Carries over unchanged: immutable versions, lineage, promotion gates with rollback, progressive rollouts, SLOs with error budgets, per-feature cost attribution, secrets handling, least privilege, audit logs.

What changes: outputs are **non-deterministic**, so one run proves little; a **vendor-hosted model can drift** without any change on your side; **token economics** make cost a function of prompt length and output length; **long context** adds new failure modes; evaluation often relies on a **judge model** that itself needs validation; and the attack surface includes **prompt injection** and data exfiltration through tools.

Artifacts to version: prompt templates, the retrieval index together with its embedding model and chunking settings, tool schemas, guardrail configs, sampling parameters and the eval sets. A release bundle (compare 5.17) should pin all of them as one unit.

Team boundary: for interviews you must be able to explain the mapping above and the changed risks; for depth on RAG, fine-tuning, agents, guardrails and LLM serving, hand off to the sibling LLMOps interview-preparation project (prompt versioning is its module 03, serving module 05, RAG evaluation module 07, guardrails module 08, tracing module 10, cost module 11, CI/CD module 13).

Edge cases: a provider **silently updates a model alias**, so pin dated or versioned snapshots where offered and run a canary eval on a schedule; an **embedding model change invalidates the index** (vectors from different models are not comparable), requiring a full re-embed and a parallel index during cutover; a **prompt change is a code change** yet often tested like config, so it needs review and an eval gate.

Cross-Connection Scenarios

- Release bundle and GitOps promotion: 5.17.
- Eval gate in CI as a variation on promotion gates: Module 04 section 3 (legacy).
- Quality monitoring without labels: Module 08 section 2 (legacy).
- Evaluating and monitoring generative output: 19.2; serving and cost: 19.3.

Production Anti-Patterns

- Treating prompts as untracked strings: nobody can tell which prompt produced a bad answer, and rollback is guesswork.
- No eval gate for prompt changes: a one-line edit silently regresses a behaviour no unit test covers.
- Assuming classical drift metrics apply to text without adaptation: PSI on raw token counts says little, so monitor embeddings, topic mix and judged quality instead.

Minimal Working Code

A prompt registry with content-hash versions, alias-based promotion and rollback, and a diff, mirroring model-registry semantics.

```
import difflib, hashlib

class PromptRegistry:
    def __init__(self):
        self.versions, self.aliases, self.history = {}, {}, {}

    def register(self, name, text):
        vid = hashlib.sha256(text.encode()).hexdigest()[:8]        # content-addressed version
        self.versions[(name, vid)] = text
        return vid

    def promote(self, name, alias, vid):
        assert (name, vid) in self.versions, "unknown version"
        self.history.setdefault((name, alias), []).append(self.aliases.get((name, alias)))
        self.aliases[(name, alias)] = vid

    def rollback(self, name, alias):
        self.aliases[(name, alias)] = self.history[(name, alias)].pop()

    def resolve(self, name, alias):
        return self.versions[(name, self.aliases[(name, alias)])]

    def diff(self, name, v1, v2):
        a, b = self.versions[(name, v1)].splitlines(), self.versions[(name, v2)].splitlines()
        return "\n".join(difflib.unified_diff(a, b, v1, v2, lineterm=""))

reg = PromptRegistry()
v1 = reg.register("summarise", "Summarise the text.\nBe concise.")
v2 = reg.register("summarise", "Summarise the text.\nBe concise.\nCite sources.")
reg.promote("summarise", "prod", v1)
reg.promote("summarise", "prod", v2)
print(reg.diff("summarise", v1, v2))
reg.rollback("summarise", "prod")
assert reg.resolve("summarise", "prod").endswith("Be concise.")
print("rolled back to", v1)
```

A study-aid mapping from an MLOps control to its LLM-app equivalent, extendable as a lookup table.

```
MAP = {
    "canary gate": ("compare judged quality, cost and latency on a traffic slice",
                    "set quality margins wide enough for judge noise"),
    "data validation": ("schema checks on retrieved context and tool outputs",
                        "treat retrieved text as untrusted input"),
    "model registry": ("registry of model snapshots, prompts, indexes, tool schemas",
                       "pin vendor snapshot ids, not floating aliases"),
    "drift monitor": ("query-embedding drift, topic mix, refusal and fallback rate",
                      "no ground truth: rely on proxies and sampled review"),
    "unit tests": ("golden-set evals with rubrics, run several times",
                   "non-determinism needs repeated runs and intervals"),
}

def equivalent(control):
    key = control.lower()
    if key not in MAP:
        return f"{control}: no entry; add one (known: {', '.join(sorted(MAP))})"
    what, caveat = MAP[key]
    return f"{control} -> {what} | caveat: {caveat}"

print(equivalent("Canary gate"))
print(equivalent("feature store"))
```

Interview Angle

Q: How would you adapt an MLOps platform to support LLM applications? **A:** Reuse the platform's registry, CI, rollout, observability and cost layers, and add: a prompt/index/tool-schema registry feeding a single release bundle, an eval service with judges and golden sets as a CI gate, trace logging with token and cost attribution, and guardrail and egress controls. Trade-off: reuse speed versus LLM-specific needs such as streaming and token-based autoscaling. Check: every production answer traces to a bundle id, eval report and cost record.

Q: What stays the same and what changes in monitoring for a RAG app? **A:** Same: SLOs on latency and errors, version labels, alerting and dashboards. Changed: no ground-truth labels, so monitor retrieval hit quality, groundedness and refusal rate by sampled judged answers, plus token cost and query-embedding drift; index freshness becomes a first-class metric. Trade-off: judge cost versus coverage. Check: judged-quality trend by slice, with the judge's agreement with human labels reported alongside.

19.2 Evaluating and Monitoring Generative Systems (Short Treatment)

Conceptual Deep Dive

Evaluation starts with a **golden set**: versioned inputs with expected behaviour, built from real traffic and failure cases, scored by a **rubric** (criteria with scales). Reference-based metrics (exact match, overlap scores) suit tasks with a single correct answer; open-ended tasks use **judge models** with explicit rubrics. A judge is a measuring instrument, so **calibrate it against human labels** and report agreement (percent agreement and Cohen's kappa) with an interval before trusting its scores.

Evaluate by **component**: retrieval (recall of the right passages), groundedness (answer supported by context), answer quality and safety. Evaluating retrieval separately from generation tells you which half to fix.

Monitoring adds to classical signals: full trace logging, tokens and cost per request, refusal and fallback rates, user feedback (thumbs, edits, re-asks) and **embedding drift on queries** to detect new topics your eval set does not cover. Latency and cost alone say nothing about whether answers got worse.

Regression control: use the eval as a CI gate that compares against a baseline with a tolerance based on run-to-run noise, keep eval sets versioned, and guard against **contamination** (eval items leaking into prompts, few-shot examples or fine-tuning data). **Online experiments** with LLM variants follow the usual A/B discipline (see 7.17) plus cost and latency guardrails.

For depth on rubrics, judge design, RAG evaluation and data flywheels, use the sibling LLMOps interview-preparation project (its RAG evaluation and evaluation-operations modules).

Treat the judge prompt as a versioned artifact too: a changed rubric or judge model shifts every score, so re-baseline when either changes and never compare scores across judge versions. Keep a small human-labelled audit slice that is re-scored regularly to catch judge drift.

Edge cases: judges show **bias towards longer answers or their own model family**, so control length and use a different family where possible; an **eval set leaking into prompts** inflates scores; **non-determinism** makes single runs unreliable, so repeat runs and report spread.

Cross-Connection Scenarios

- Eval gating in CI as a promotion gate: 19.1 and Module 05 section 2 (legacy).
- Interval and sample-size reasoning: 7.17.
- Quality monitoring without labels: Module 08 section 2 (legacy).
- Cost guardrails: 19.3.

Production Anti-Patterns

- Single-run eval with temperature above zero: a lucky or unlucky sample decides whether a release ships.
- Judge never validated against humans: you optimise against a metric whose relationship to quality is unknown.
- Monitoring only latency and cost: a retrieval bug or model update degrades answers while dashboards stay green.

Minimal Working Code

Percent agreement and Cohen's kappa between a stub judge and human labels, with a bootstrap interval.

```
import numpy as np

def kappa(a, b):
    po = (a == b).mean()
    pe = sum((a == k).mean() * (b == k).mean() for k in (0, 1))
    return (po - pe) / (1 - pe)

rng = np.random.default_rng(0)
n = 300
human = (rng.random(n) < 0.6).astype(int)
judge = np.where(rng.random(n) < 0.85, human, 1 - human)       # stub judge: agrees ~85%
boots = []
for _ in range(1000):
    idx = rng.integers(0, n, n)
    boots.append(kappa(human[idx], judge[idx]))
lo, hi = np.percentile(boots, [2.5, 97.5])
print(f"agreement {(human == judge).mean():.3f}, kappa {kappa(human, judge):.3f}, "
      f"95% CI [{lo:.3f}, {hi:.3f}]")
```

An eval gate that runs a dataset through a callable several times, aggregates pass rates and compares with a baseline given a tolerance.

```
import random
import statistics

def eval_gate(system, dataset, baseline, tolerance=0.03, runs=3):
    rates = []
    for r in range(runs):
        passed = sum(system(item, seed=r) == item["expected"] for item in dataset)
        rates.append(passed / len(dataset))
    mean = statistics.mean(rates)
    return {"rates": [round(x, 3) for x in rates], "mean": round(mean, 3),
            "spread": round(max(rates) - min(rates), 3),
            "passed": mean >= baseline - tolerance}

def stub_system(item, seed):
```

```
    rng = random.Random(f"{item['id']}-{seed}")          # non-deterministic per run, repeatabl
e
    return item["expected"] if rng.random() < 0.9 else "wrong"

data = [{"id": i, "expected": "ok"} for i in range(200)]
print(eval_gate(stub_system, data, baseline=0.90))
print(eval_gate(stub_system, data, baseline=0.97))
```

Interview Angle

Q: How do you gate a prompt change in CI? **A:** Run the versioned golden set against the candidate and baseline prompts, several runs each, with sampling settings fixed and judged by a validated judge. Block if the candidate's mean falls below baseline minus a tolerance that exceeds measured run-to-run noise, or if any safety or format check fails. Trade-off: eval cost and CI time versus protection. Check: gate decisions are stable across reruns and the judge agrees with humans (report kappa).

Q: What would you monitor in production for a retrieval-augmented assistant? **A:** Operational: latency, errors, tokens, cost per request. Quality: sampled judged groundedness and answer quality, retrieval hit proxies, refusal and fallback rate, user feedback signals. Data: query-embedding drift and index freshness. Trade-off: judge spend versus sampling rate. Check: alert on a sustained drop in judged groundedness on a fixed random sample, with segment breakdowns by query type and index version.

19.3 Serving and Cost for Large Models on a Classical ML Platform

Conceptual Deep Dive

Large-model serving differs from classical serving in what limits it. GPU memory is dominated by **weights** plus the **KV cache** that grows with context length and concurrent sequences. Decoding is **memory-bandwidth-bound**, since each new token reads the weights, so throughput comes from **continuous batching** of many sequences. Responses are **streamed**, so time to first token and inter-token latency matter more than total time.

Serving options: managed model APIs, or self-hosting on engines such as vLLM, TGI or Triton-class servers (check current docs). Build versus buy depends on volume (self-hosting pays back only at sustained utilisation), privacy and data-residency constraints, latency control and the team's ability to run GPUs. The sibling LLMOps project covers engines in depth.

Cost model: cost is roughly tokens in plus tokens out times price per token (verify against current pricing); levers are prompt trimming, caching of repeated prefixes or responses, **routing** easy requests to a smaller model, and batching offline workloads off-peak. Attribute cost per feature and per tenant from the start.

Reliability: provider rate limits require retries with exponential backoff and jitter under a retry budget, **fallbacks** across models or providers, timeouts, and cancellation of streams when the client disconnects.

Security and compliance: data egress to vendors, PII redaction before sending, retention and audit logs, and contractual data-use terms.

Capacity math (estimate): weights bytes = parameters x bytes per parameter. KV cache bytes per token = 2 x layers x KV heads x head dim x bytes per element; total KV = per-token bytes x context length x concurrent sequences. Add headroom for activations, fragmentation and runtime overhead.

Edge cases: **retry storms** against rate limits amplify the problem unless retries share a budget; **cost spikes from prompt growth** (more history, bigger retrieved context) appear without any traffic change; **tail latency from long generations** means max output tokens and timeouts must be bounded.

Cross-Connection Scenarios

- Tail latency, load shedding, retry budgets: 6.17.
- Quantization and optimisation levers: 6.16.
- GPU scheduling and sharing: Module 09 sections 2-3 (legacy).
- Egress, PII and audit evidence: Module 10 (legacy).

Production Anti-Patterns

- No per-feature cost attribution: a single feature quietly consumes most of the budget and nobody owns it.

- Single provider with no fallback: a provider outage or quota change becomes your outage.
- Sending raw PII to a third-party API: it creates a compliance exposure that cannot be undone after the fact.

Minimal Working Code

A weight and KV-cache memory estimator (an estimate only; the config below is hypothetical and excludes activations and runtime overhead).

```
import math

def estimate_memory(params_b, bytes_per_param, layers, kv_heads, head_dim,
                    context, concurrent, kv_bytes=2, headroom=1.2):
    weights = params_b * 1e9 * bytes_per_param
    kv_per_token = 2 * layers * kv_heads * head_dim * kv_bytes      # K and V
    kv_total = kv_per_token * context * concurrent
    total = (weights + kv_total) * headroom
    return {"weights_gib": round(weights / 2**30, 1),
            "kv_per_token_kib": round(kv_per_token / 2**10, 1),
            "kv_total_gib": round(kv_total / 2**30, 1),
            "total_gib_with_headroom": round(total / 2**30, 1)}

cfg = dict(params_b=7, layers=32, kv_heads=8, head_dim=128, context=4096, concurrent=16)
for name, bpp in [("fp16", 2), ("int8", 1)]:
    est = estimate_memory(bytes_per_param=bpp, **cfg)
    gpus = math.ceil(est["total_gib_with_headroom"] / 80)          # assume 80 GiB devices
    print(name, est, "-> GPUs needed (estimate):", gpus)
```

A cost-aware router that sends requests to a small or large model by a noisy difficulty score, tracking cost savings, escalation rate and quality in a simulation.

```
import random

def simulate(threshold, n=5000, seed=0, c_small=1.0, c_large=15.0):
    rng = random.Random(seed)
    cost = correct = escalated = 0
    for _ in range(n):
        difficulty = rng.random()
        score = min(1.0, max(0.0, difficulty + rng.gauss(0, 0.1)))   # heuristic is noisy
        if score > threshold:
            escalated += 1
            cost += c_small + c_large                                # small model tried first
            correct += rng.random() < 0.95
        else:
            cost += c_small
            correct += rng.random() < 1 - 0.5 * difficulty
    return cost / n, correct / n, escalated / n

all_large_cost, all_large_acc = 15.0, 0.95
for t in (0.3, 0.5, 0.7, 1.1):
    c, acc, esc = simulate(t)
    print(f"threshold {t}: cost {c:5.2f} ({1 - c / all_large_cost:5.1%} saved) "
          f"acc {acc:.3f} (all-large {all_large_acc}) escalation {esc:.1%}")
```

Interview Angle

Q: How do you estimate GPUs needed to serve an LLM feature at 50 requests/s? **A:** Start from the load: tokens per request in and out give tokens/s, and concurrency follows from Little's law (rate x time per request). Compute memory per replica (weights plus KV cache for that concurrency) to see whether it fits, then benchmark throughput of your engine on representative prompts to get requests/s per GPU. Replicas = load / measured capacity, with headroom. Trade-off: utilisation versus tail latency. Check: load test p99 time to first token at target rate.

Q: How would you cut LLM inference spend 40% without hurting quality? **A:** Attribute cost per feature, then pull levers in order of safety: trim prompts and cap output tokens, cache repeated prefixes and answers, route easy requests to a smaller model, batch non-interactive work, and consider quantization for self-hosted models. Trade-off: each lever risks quality or latency. Check:

golden-set quality and judged win-rate unchanged within noise, while cost per request and escalation rate are tracked in production.